

RETI SEQUENZIALI

- RETI COMBINATORIE → HANNO N INGRESSI E M USCITE, E L'USCITA IN OGNI ISTANTE DIPENDE SOLO DAI VALORI CORRENTI DEGLI INGRESSI
- RETE SEQUENZIALE → HANNO ANCH'ESSE N INGRESSI ED M USCITE, MA L'USCITA DIPENDE DA:
 - Dai valori degli ingressi nell'istante corrente
 - Dai valori che gli ingressi (o le uscite) hanno assunto in istanti precedenti } HANNO MEMORIA INTERNA
- MEMORIA INTERNA LIMITATA
 - PER REALIZZARE LA PARTE "MEMORY", SI AGGIUNGE RETROAZIONE (FEEDBACK) CHE RE-INIETTA SULL'INGRESSO UNA QUALCHE COMBINAZIONE DELLE USCITE
 - POICHÉ LA MEMORIA È COSTITUITA DA UN NUMERO FINITO DI ELEMENTI (BIT), LA RETE SEQUENZIALE PUÒ MEMORIZZARE SOLO UN NUMERO FINITO DI "STATI PASSATI"

ESEMPIO DI RETE SEQUENZIALE

● DUE PORTE NOR COLLEGATE IN RETROAZIONE INCROCIATA: L'USCITA DI CIASCUNA TORNA IN UNO DEGLI INGRESSI DELL'ALTRA. OLTRE AI DUE INGRESSI DI COMANDO A E B, LE USCITE SONO INDICATE C (SET) E D (RESET)

● SE NETTO A=1 (E B=0)

- LA PRIMA NOR HA IN INGRESSO UN 1, QUINDI LA SUA USCITA C DIVENTA 0
- QUEL "0" VIENE RE-INIETATO NELL'ALTRA NOR, E DI CONSEGUENZA D (LA SUA USCITA) DIVENTA 1

● SE NETTO B=1 (E A=0) → SIMMETRICAMENTE OTTENGO C=1 E D=0, CIÒ È IL LATCH SI "SETTA"

● SE A=B=0 NÈ SI SETTA NÈ SI RESETTA: IL LATCH MANTIENE LO STATO PRECEDENTE GRAZIE ALLA RETROAZIONE

● SE A=B=1 ENTRAMBE LE NOR FORZANO C=D=0: UNO STATO GENERALMENTE NON VALIDO NEL LATCH (CONDIZIONE PROIBITA)

PERCHÉ È SEQUENZIALE

- LE USCITE C, D DIPENDONO NON SOLO DAI COMANDI A, B ATTUALI, MA ANCHE DAL VALORE CHE GLI STESSI C, D AVEVANO POCO PRIMA
- LA PRESENZA DELLA RETROAZIONE È QUÒ CHE INTRODUCE LA CAPACITÀ DI "RICORDARE" UNO STATO

Elemento di memoria: il latch. Questo circuito è il più semplice blocco di memoria bistabile. Viene chiamato SR-latch (Set-Reset latch). È in grado di memorizzare 1 bit finché non riceve un comando di set o reset

DEFINIZIONE DI UNA RETE SEQUENZIALE

- INSIEME DEGLI STATI DI INGRESSO → È SEMPLICEMENTE L'INSIEME DI TUTTE LE CONFIGURAZIONI POSSIBILI DELLE VARIABILI BOOLEANE CHE LA RETE RICEVE IN INGRESSO (SE HO N INGRESSI, CI SONO 2^N POSSIBILI COMBINAZIONI)
- INSIEME DEGLI STATI INTERNI → OGNI STATO INTERNO RAPPRESENTA UNA POSSIBILE "STORIA PASSATA" DELLA RETE, OSSIA UNA CONFIGURAZIONE PRECEDENTE DELLE USCITE (O DEI FLIP-FLOP (LATCH) CHE LA RETE PUÒ TROVARSI A MEMORIZZARE. ANCHE QUI, SE HO K BIT DI MEMORIA, ESISTONO 2^K STATI INTERNI
- INSIEME DEGLI STATI DI USCITA → È L'INSIEME DI TUTTE LE CONFIGURAZIONI POSSIBILI DELLE VARIABILI BOOLEANE DI USCITA (PER M USCITE, 2^M COMBINAZIONI)
- DIAGRAMMA DEGLI STATI → È LA RAPPRESENTAZIONE GRAFICA (O TABELLARE) DI COME EVOLUE IL SISTEMA:
 - SI INDICA LO STATO INTERNO SUCCESSIVO
 - E SI INDICANO LE USCITE PRODOTTE IN QUELL'ISTANTE

AUTOMA A STATI FINITI

LA DEFINIZIONE DATA COINCIDE ESATTAMENTE CON QUELLA DI UN AUTOMA A STATI FINITI (FINITE-STATE MACHINE) IN TEORIA DEI LINGUAGGI FORMALI:

- STATI INTERNI ↔ STATI DELL'AUTOMA
- INGRESSI ↔ SIMBOLI DEL "LINGUAGGIO"
- USCITE ↔ AZIONI O SIMBOLI ENMESSI (NEARLY O MOORE)
- DIAGRAMMA DEGLI STATI ↔ TRANSIZIONI DELL'AUTOMA

DIAGRAMMA DEGLI STATI

● DIAGRAMMA DEGLI STATI (MODELLO DI MEALY)

- COS'È - UN GRAFO ORIENTATO IN CUI
 - NODI = STATI INTERNI DELLA MACCHINA
 - ARCHI = TRANSIZIONI DI STATO, ETICHETTA CON "INGRESSO/USCITA"
- COME LEGGERLO - SE LA MACCHINA SI TROVA NELLO STATO S_i E RICEVE UN BIT DI INGRESSO X, SEQUE L'ARCO CHE PARTE DA S_i CON ETICHETTA X/Y. GIUNTA ALLO STATO S_j , PRODUCE IN USCITA IL VALORE Y.

STATO CORRENTE	INGRESSO X	STATO SUCCESSIVO	USCITA Y	COMMENTO
S_0	0	S_0	0	RESTO IN S_0 , NON INIZIO NESSUNA "1"
S_0	1	S_1	0	INIZIO POSSIBILE SEQUENZA ("1")
S_1	1	S_1	0	RESTO IN "HO LETTO 1" (NÙ 1 NON MI AVANZA)
S_1	0	S_2	0	HO "10" PASSA A S_2
S_2	0	S_0	0	"100" NON SERVE, RIFORMO DA CAPO
S_2	1	S_3	0	HO "101", VADO A S_3
S_3	1	S_1	0	"1011" NON COMPLETA, MA QUEST'ULTIMO 1 PUÒ AVERE INIZIO DI NUOVA SEQUENZA → S_1
S_3	0	S_4	1	HO COMPLETATO "1010" ENETO 1 E VADO A S_4
S_4	0	S_0	0	DOPO IL RICONOSCIMENTO "1010", O NON RIPARTE LA SEQUENZA, TORNA A S_0
S_4	1	S_1	0	DOPO "1010" + 1, QUELL'1 VALE COME INIZIO, VADO A S_1

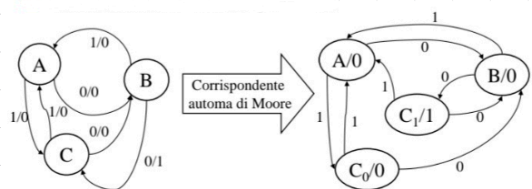
Logica del riconoscitore

- Avanzamento → Ogni stato "ricorda" quanti bit della sequenza target (1010) ho già visto in successione, senza perdita di sovrapposizioni utili.
- Emissione di 1 → L'unica transizione con uscita 1 è: significa "dopo aver letto 101, se adesso leggo 0, ho riconosciuto 1010".
- Gestione dell'overlap → Notate che da S_4 , se invece leggo "1" passo a mantenendo quell'1 come possibile inizio di un nuovo "1-0-1-0".

Analogamente, da su ingresso "1" torno a S_1 , perché quell'1 può già essere il primo bit di un'altra occorrenza

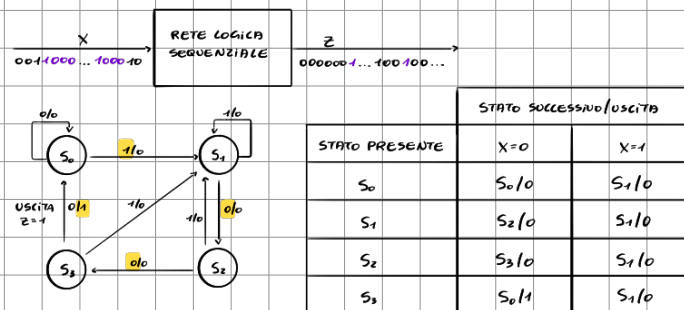
Diagramma di Mealy e di Moore

- MEALY → NELLO SCHEMA DI SINISTRA, OGNI ARCO PORTA L'INFORMAZIONE "SE ARRIVO VIA X, EFFETTO Y"
- MOORE → NELLO SCHEMA DI DESTRA, CIASUN NODO È ETICHETATO CON LO STATO E CON L'USCITA Y CHE DEVO RESTITUIRE IN QUEL STATO, MENTRE SEI ARCHI, PORTANO SOLO IL VALORE DELL'INGRESSO X
- ES. LO STATO C_1 IN MOORE È ETICHETATO $C_1/1$, SIGNIFICA "QUANDO SONO IN C_1 , L'USCITA VALE 1"
- SE DA $A/0$ (USCITA=0) RICEVO UN 1, VADO IN $C_1/1$ E L'USCITA DIVENTA 1 SOLO AL CAMBIO DI STATO



ESEMPIO DI DIAGRAMMA DEGLI STATI

SI VOGLIA PROGETTARE UNA RETE LOGICA IN GRADO DI RICONOSCERE IN UNA SEQUENZA DI BIT IN INGRESSO LA STRINGA **1000** PONENDO A 1 L'USCITA Z SOLO QUANDO SI ABBIA IL RICONOSCIMENTO DI TALE STRINGA



IMPLEMENTAZIONE DELLO STATO DI UNA RETE SEQUENZIALE

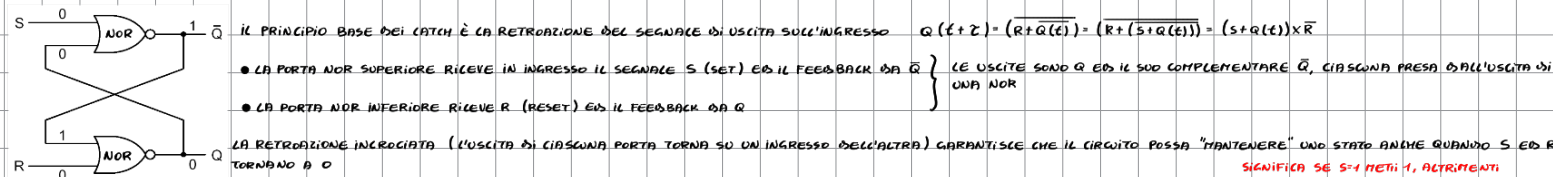
- STATO INTERNO → CONFIGURAZIONE DI BIT
- OGNI STATO SIMBOLICO S_i DELLA MACCHINA SEQUENZIALE VA RAPPRESENTATO DA UN INSIEME DI VARIABILI BOOLEANE DI "STATO"
- QUANTI BIT SERVONO? (TIPICAMENTE BIT DI LATCH O FLIP-FLOP)
- SE LA MACCHINA HA 5 STATI, SERVONO ALMENO 3 BIT TALI CHE $2^3 \geq 5$
- ESEMPIO → PER LA SEQUENZA "1000" ABBIAMO 4 STATI S_0, S_1, S_2, S_3 CON $n=2$ BIT POSSIBILI CODIFICARLI COSÌ: $S_0=00 \quad S_1=01 \quad S_2=10 \quad S_3=11$

STATO PRESENTE	STATO SUCCESSIVO/USCITA	
	X=0	X=1
00	00/0	01/0
01	10/0	01/0
10	11/0	01/0
11	00/1	01/0

ELEMENTI DI MEMORIA

- Come visto lo stato di una rete sequenziale rappresentato da un certo numero di bit è
- Vediamo ora dei semplici elementi di memoria, ognuno dei quali può memorizzare un bit
- L'unione di più elementi di memoria da 1 bit consentiti memorizzare i possibili stati assunti da una rete sequenziale
- Gli elementi di memoria si dividono in due categorie:
 - elementi di memoria asincroni e sincroni (latch)
 - elementi di memoria "master-slave" (flip flop)

IL LATCH SET-RESET ASINCRONO (SR)



- EQUAZIONE DI TRANSIZIONE ⇒ PARTENDO DALLA PORTA NOR INFERIORE, SI PUÒ DERIVARE L'EQUAZIONE DEL PROSSIMO VALORE DI Q
- MA UTILIZZANDO LE PROPRIETÀ DELLE NEGAZIONI E CONSIDERANDO ANCHE L'EFFETTO DI S (VIA LA NOR SUPERIORE), SI OTTIENE LA FORMA PIÙ COMPLETA

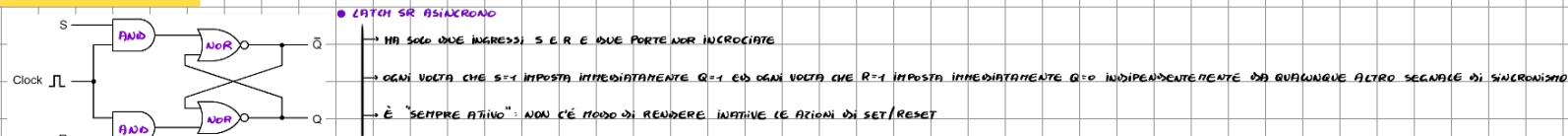
S	R	Q(t)	Q(t+1)	
0	0	0	0	MANTIENE 0
0	0	1	1	MANTIENE 1
0	1	0	0	RESET (a 0)
0	1	1	0	RESET → 0
1	0	0	1	SET → 1
1	0	1	1	SET (a 1)
1	1	0	—	NON AMMESSO
1	1	1	—	NON AMMESSO

- SE $S=1 \quad Q(t+1)=1$ S=SET SIGNAL
- SE $R=1 \quad Q(t+1)=0$ R=RESET SIGNAL
- SE $S=R=0 \quad Q(t+1)=Q(t)$ STATO INVARIATO
- SE $S=R=1$ → CONFIGURAZIONE NON AMMESSA

SIGNIFICA SE $S=1$ METTI 1, ALTRIMENTI CONSERVA $Q(t)$

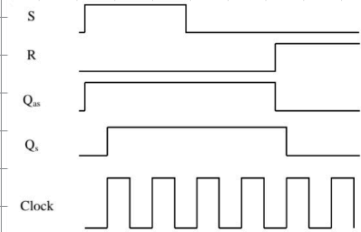
MOLTIPLICATO PER R PERCHÉ SE $R=1$ PORTO RESET A 0

IL LATCH SR ASINCRONO



AGGIUNGERE IL CLOCK

- PER RENDERLO SINCRONO, SI PONGONO DUE PORTE AND IN CASCATA VERSO CIASCUN INGRESSO S E R, CONTROLLATE DAL SEGNALE DI CLOCK
- SOLO QUANDO IL CLOCK È ALTO I SEGNALE S O R EFFETTIVAMENTE PASSANO ALLE NOR SOTTO STANTI
- QUANDO IL CLOCK È BASSO GLI INGRESSI S E R "VENGONO BLOCCATI" ED IL LATCH MANTIENE LO STATO INDIPENDENTEMENTE DAI CAMBIAMENTI DI S/R



S → SEGNALE DI SET
R → SEGNALE DI RESET
Q_n → USCITA DEL LATCH ASINCRONO (SENZA CLOCK)
Q_s → USCITA DEL LATCH SINCRONO (CON CLOCK)
CLOCK

APPENA S SALE AD 1, ANCHE Q_n PASSA A 1, INDIPENDENTEMENTE DAL CLOCK } NON TIENE CONTO DEL CLOCK SEGRE SEMPRE S/R
QUANDO R SALE, Q_n VA SUBITO A ZERO

IGNORA S E R QUANDO IL CLOCK È BASSO (LATCH "CHIUSO"/"OPAQUE"): L'USCITA RESTA FISSA
QUANDO IL CLOCK È ALTO (LATCH "APERTO"/"TRANSPARENT"), S/R AGISCONO E Q_s SEGUE S/R ESATTAMENTE COME IL LATCH ASINCRONO
APPENA IL CLOCK TORNA BASSO, Q_s "CONGELA" IL VALORE RAGGIUNTO E NON CAMBIA PIÙ ANCHE SE S/R SI MODIFICANO

LATCH JK SINCRONO

IL LATCH JK SINCRONO È UN'EVOUZIONE DEL LATCH SR SINCRONO CHE RISOLVE IL PROBLEMA DELLA CONDIZIONE PROIBITA $S=R=1$, PERMETTENDO INVECE L'INVERSIONE DELL'USCITA QUANDO $J=K=1$

J	K	Q(t)	Q(t+1)	COMPORTAMENTO
0	0	0	0	MANTIENE LO "0"
0	0	1	1	MANTIENE L'1
0	1	0	0	RESET (PORTA A 0)
0	1	1	0	RESET
1	0	0	1	SET (PORTA A 1)
1	0	1	1	SET
1	1	0	1	INVERTENTE
1	1	1	0	INVERTENTE

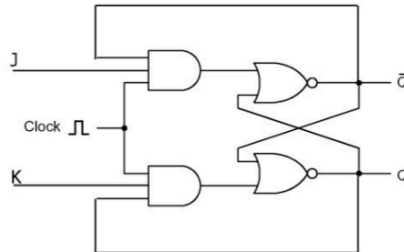
TABELLA DI VERITÀ COMPLETA

Q(t)	Q(t+1)	J	K
0	0	0	0
0	1	1	0
1	0	0	1
1	1	1	1

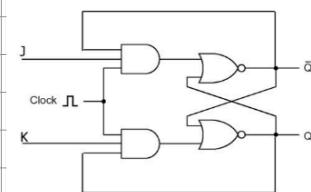
DON'T CARE

$$Q(t+1) = \bar{Q}(t)$$

CIRCUITO LOGICO DEL LATCH JK SINCRONO



$J=K=0$ → IL LATCH MANTIENE IL VALORE CORRENTE (COME IN SR CON $S=R=0$)
 $J=1, K=0$ → IL LATCH VIENE SETTATO ($Q \rightarrow 1$)
 $J=0, K=1$ → IL LATCH VIENE RESETTATO ($Q \rightarrow 0$)
 $J=K=1$ → IL LATCH INVERTE LO STATO CORRENTE (INVERTENTE)



- INGRESSI J E K PASSANO CIASCUNO ATRAVVERSO UNA PORTA AND CONTROLLATA DAL CLOCK, ESATTAMENTE COME NEL LATCH SR SINCRONO
 - LE USCITE DI QUESTE AND (ABILITATE SOLO QUANDO IL CLOCK È "APERTO") VANNO AI DUE INGRESSI "SET" E "RESET" DELLE PORTE NOR INCROCIATE
 - FEEDBACK: LE USCITE Q E $\bar{Q}$ RITORNANO SUI RISPETTIVI INGRESSI AND PER CONSENTIRE L'INVERTENTE QUANDO $J=K=1$
- QUANDO $J=K=1$ ED IL CLOCK È ATTIVO, ENTRAMBE LE AND LASCIANO PASSARE UN "1" VERSO LE NOR → CIASCUNA NOR RICEVE "1" DA J/K ED UN FEEDBACK DALL'ALTRA USCITA → AL RILASCIO DEL CLOCK LE NOR SI STABILIZZANO NEL NUOVO STATO INVERTITO

ALTRI LATCH SINCRONI

LATCH D (DELAY)

- È UN LATCH JK CON $J=\bar{R}=0$
- COMPORTAMENTO → AL FRONTE ABILITANTE DEL CLOCK, L'USCITA Q ASSUME IL VALORE DELL'INGRESSO D

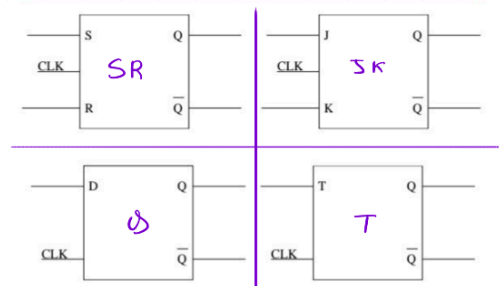
D	Q(t)	Q(t+1)
0	0	0
0	1	0
1	0	1
1	1	1

LATCH T (TRIGGER)

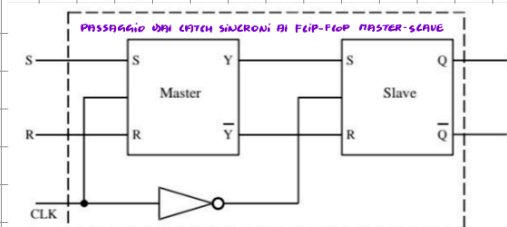
- È UN LATCH JK CON $J=K=T$
- COMPORTAMENTO: AL FRONTE ABILITANTE DEL CLOCK, SE $T=1$ IL LATCH INVERTE LO STATO CORRENTE, SE $T=0$ LO MANTIENE

$$Q(t+1) = T \oplus Q(t)$$

T	Q(t)	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

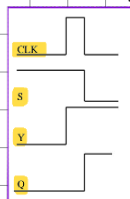


ELEMENTI DI MEMORIA "MASTER-SLAVE"



DISACCOPPIAMENTO DEI TEMPI

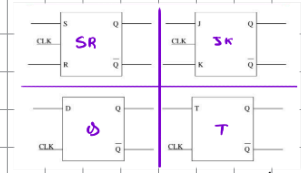
- FASE DI SALITA DEL CLOCK (0 → 1)
MASTER: ABILITATO, LEGGE S/R ED AGGIORNA LA SUA USCITA INTERMEDIA Y
SLAVE: DISABILITATO, NON SEDE IL MASTER; IL VECCHIO Q RESTA STABILE
- FASE DI DISCESA DEL CLOCK (1 → 0)
MASTER: DISABILITATO, TRATTIENE IL VALORE Y SENZA PIÙ AGGIORNARLO ANCHE SE S/R CAMBIANO
SLAVE: ABILITATO (CLOCK INVERTITO), LEGGE Y E LO TRASFERISCE SU Q



QUESTO MECCANISMO GARANTISCE CHE:

- IL DATO VENGA CARICATO IN UN UNICO ISTANTE (ALLA SALITA DEL CLOCK), EVITANDO GLITCH DOVUTI A VARIAZIONI ASINCRONE DI S/R
- IL DATO VENGA RILASCIATO SULL'USCITA IN UN ALTRO ISTANTE PRECISO (ALLA DISCESA DEL CLOCK), FORNENDO STABILITÀ E TIMING DETERMINISTICO

SENZA TRIANGOLO → LATCH SINCRONO (LEVEL-SENSITIVE)



IL CLOCK IN QUESTO CASO È "SEMPLICEMENTE" UNA ABILITAZIONE DI LIVELLO

LATCH SINCRONO → SI APRE E SI CHIUDE IN FUNZIONE DEL LIVELLO DEL CLOCK

● UN LATCH SINCRONO È UN DISPOSITIVO "LEVEL-SENSITIVE"

- QUANDO IL CLOCK È AL LIVELLO ABILITANTE (TIPICAMENTE ALTO), IL LATCH È "APERTO" (TRANSPARENT), CIOÈ L'INGRESSO (SR, JK, O T) VIENE CONTINUAMENTE TRASFERITO SULL'USCITA
- QUANDO IL CLOCK TORNA AL LIVELLO DI DISABILITAZIONE, IL LATCH SI "CHIUDE" E TRATTIENE IL VALORE FINCHÉ IL CLOCK NON TORNA NUOVAMENTE ABILITATO

ARCHITETTURA DI UNA RETE SEQUENZIALE



X, S, Y SONO I VETTORI DELLE VARIABILI DI INGRESSO, DI STATO E DI USCITA

- RETE COMBINATORIA PER LA TRANSIZIONE DI STATO → PRENDE IN INPUT X E L'ATTUALE STATO INTERNO S, E CALCOLA IL PROSSIMO STATO S'. REALIZZA LE EQUAZIONI DELLO STATO SUCCESSIVO $S' = f(X, S)$
- BLOCCO RITARDBANTE → È IL "CUORE" DELLA MEMORIA: TRATTIENE S' FINO AL PROSSIMO EVENTO DI SINCRONISMO (IN UN FLIP-FLOP, AVVIENE AL FRONTE DI CLOCK). IN USCITA FORNISCE IL VERO STATO INTERNO S
- RETE COMBINATORIA PER LE USCITE → USA X E L'ATTUALE S' PER CALCOLARE LE USCITE Y. REALIZZA LE EQUAZIONI DI USCITA $Y = g(X, S)$
- FEEDBACK → L'USCITA Y DEL BLOCCO RITARDBANTE RIENTRA NELLA RETE DI TRANSIZIONE DI STATO, CHIUDENDO IL CICLO DI RETROAZIONE

TIP DI RETE SEQUENZIALE

	ASINCRONA	SINCRONA
BLOCCO RITARDBANTE	ASSENTE	PRESENTE, INTERVALLO DI TEMPO REGOLARI
EVOLOZIONE	GLI INGRESSI E GLI STATI INTERNI CAMBIANO IMMEDIATAMENTE FINCHÉ IL CIRCUITO NON RAGGIUNGE UNO STATO STABILE	GLI INGRESSI E GLI STATI VENGONO "CAMPIONATI" SOLO IN CORRISPONDENZA DI EVENTI REGOLARI (TIPICAMENTE IL FRONTE DI UN CLOCK)
PROGETTAZIONE	DIFFICILE PROGETTAZIONE, SI USANO PER LA COMUNICAZIONE FRA CIRCUITI SINCRONIZZATI INDIPENDENTEMENTE L'UNO DALL'ALTRO	IL BLOCCO RITARDBANTE È COSTITUITO DA ELEMENTI DI MEMORIA SINCRONIZZATI CON RETOURA DEI SEGNALE DI INGRESSO

PERCHÉ PREFERIRE LE RETI SINCRONE

- TIMING PREDEICIBILE → TUTTE LE TRANSIZIONI DI STATO AVVENGONO SU UN FRONTE DI CLOCK COMUNE
- MODULARITÀ → FACILI DA CONNETTERE
- SINCROEITÀ → I BLOCCHI RITARDBANTI CATTURANO STATI STABILI, EVITANDO COMPORTAMENTI INDEFINITI TRA LE PORTE COMBINATORIE

RETI SEQUENZIALI SINCRONE

● LETURA A INTERVALLI REGOLARI

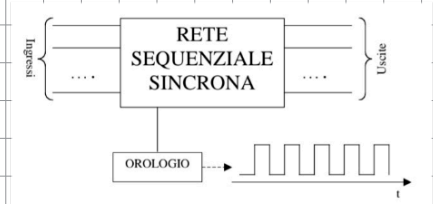
UNA RETE SEQUENZIALE SINCRONA NON REAGISCE IMMEDIATAMENTE AD OGNI CAMBIAMENTO DEGLI INGRESSI, MA LI "CAMPIONA" SOLO IN ISTANTI PRESTABILITI ED EQUIDISTANZIATI NEL TEMPO

● CLOCK (OROLOGIO)

È UN GENERATORE DI IMPULSI PERIODICI CHE SCANDISCE QUESTI ISTANTI DI CAMPIONAMENTO. OGNI IMPULSO FA PARTIRE UNA NUOVA ELABORAZIONE

● FUNZIONAMENTO

- ALL'IMPULSO DI CLOCK, LA RETE LEGGE TUTTI GLI INGRESSI SIMULTANEAMENTE
- SULLA BASE DI QUESTI INGRESSI E DELLO STATO INTERNO CORRENTE, CALCOLA LE NUOVE USCITE E LO STATO SUCCESSIVO
- NELLO STATO SUCCESSIVO VIENE MEMORIZZATO NEL REGISTRO INTERNO, SOSTITUENDO IL VECCHIO STATO



IL "BLOCCO RITARDBANTE"

● COS'È → UN REGISTRO, CIOÈ UN INSIEME DI FLIP-FLOP CHE IMMAGIZZINANO I BIT DI STATO DELLA RETE

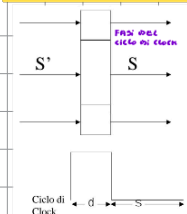
● PERCHÉ "RITARDBANTE" → RALLENTA (SINCRONIZZA) IL PASSAGGIO DELLO STATO SUCCESSIVO AL CURRENT-STATE AL SOLO ISTANTE DEL CLOCK, EVITANDO CAMBIAMENTI ASINCRONI TRA LE PORTE COMBINATORIE

● RUOLI FONDAMENTALI

- MEMORIZZAZIONE → AD OGNI FRONTE DI CLOCK, IL REGISTRO CARICA LO STATO SUCCESSIVO S' CALCOLATO DALLA LOGICA COMBINATORIA DELLA TRANSIZIONE DI STATO. MANTIENE QUEL VALORE S FINO AL PROSSIMO FRONTE DI CLOCK
- SINCRONIZZAZIONE → GRAZIE AI FLIP-FLOP (MASTER-SLAVE), TUTTE LE RICHE DI STATO VENGONO AGGIORNATE SIMULTANEAMENTE, GARANTENDO UN TIMING DETERMINISTICO. LE RETI COMBINATORIE A MONTE POSSONO STABILIZZARSI TRA UN FRONTE DI CLOCK E L'ALTRO, SENZA INDURRE CLITCH SUL REGISTRO
- DETERMINISMO → IL CONTENUTO DEL REGISTRO DOPO CIASCUN FRONTE DI CLOCK È UNO DEGLI STATI CONSENTITI DELLA MACCHINA A STATI FINITI. IN QUESTO MODO, LA RETE SEQUENZIALE EVOLUE PASSO-PASSO ATTRAVERSO GLI STATI, UNO PER OGNI IMPULSO DI CLOCK
- IL REGISTRO È IL CUORE DELLA RETE SEQUENZIALE SINCRONA: GRAZIE AD ESSO OGNI IMPULSO DI SINCRONISMO PERMETTE L'ELABORAZIONE DEI DATI PRESENTI IN INGRESSO.

Il contenuto del registro è infatti uno degli S staticoconsentiti, ed è rappresentato da configurazione rappresentarlo degli N bit necessari unaper

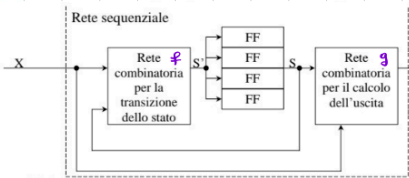
DISACCOPIAMENTO FRA CARICAMENTO DELLO STATO E CALCOLO DELLO STATO SUCCESSIVO



- PERIODO ϕ (LATO FRONTE DI SALITA) → EVENTO DI CLOCK: AL FRONTE DI SALITA IL REGISTRO (BLOCCO RITARDBANTE) CARICA IL VALORE S' CHE LA LOGICA COMBINATORIA HA PREPARATO. IN QUEL BREVE ISTANTE, IL NUOVO STATO DIVENTA IMMEDIATAMENTE LO STATO CORRENTE S
- PERIODO σ (TRA FRONTE DI SALITA E IL SUCCESSIVO) → CON IL REGISTRO "BLOCCATO" SUL VALORE S, LA RETE COMBINATORIA VEDERE S COSTANTE (E LEGGE ANCHE GLI INGRESSI X CORRENTI) E CALCOLA LE USCITE $Y = g(X, S)$ E LO STATO SUCCESSIVO $S' = f(X, S)$

NOTA: GLI INGRESSI X POSSONO VARIARE LIBERAMENTE SOLO DURANTE σ , PERCHÉ DURANTE ϕ LA RETE È IMPEGNATA A TRASFERIRE S' NEI FLIP-FLOP

IMPLEMENTAZIONE DEL BLOCCO RITARDA



IL BLOCCO RITARDA È SEMPLICEMENTE UN REGISTRO COMPOSTO DA TANTI FLIP-FLOP QUANTI SONO I BIT DI STATO S

OGNI FLIP-FLOP È UN MASTER-SLAVE CHE:

- ALLA SALITA DEL CLOCK CAMPIONA IL PROPRIO INGRESSO (IL CORRISPONDENTE BIT DI S) E LO MEMORIZZA
- NEL RESTO DEL CICLO FORNISCE QUEL VALORE STABILE COME BIT DI STATO S ALLA LOGICA COMBINATORIA

COMBINA F: LOGICA COMBINATORIA PER LO STATO SUCCESSIVO COMBINA G: LOGICA COMBINATORIA PER LE USCITE

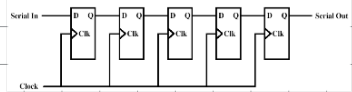
I FLIP-FLOP FORMANO IL REGISTRO CHE "RITARDA" (SINCRONIZZA) IL PASSAGGIO DA S' A S AL SOLO FRONTE DI CLOCK

Perché è importante

- Prevedibilità: tutte le parti combinatorie vedono stati e ingressi stabili durante la fase di calcolo o, eliminando glitch
- Sincronismo centrale: un'unica onda di clock governa il passaggio tra stati, rendendo più semplice il timing in sistemi complessi
- Modularità: il registro può essere scalato (più bit) senza modificare la logica di calcolo; facilitando la progettazione a blocchi.

VANTAGGI DEL DISACCOPIAMENTO

- Realizza l'effetto "ritardante", in quanto, mentre viene presentato in ingresso lo stato successivo, viene contemporaneamente letto in uscita lo stato precedente
- Il nuovo stato viene memorizzato soltanto nel periodo in cui i segnali si sono stabilizzati
- Non ci possono essere variazioni dello stato indesiderate, in quanto il funzionamento è sincrono: forza una sola variazione di stato per ogni impulso



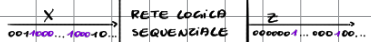
ESEMPIO DI REGISTRO CON FLIP-FLOP: IL REGISTRO "A SCORRIMENTO" (SHIFT-REGISTER)

AD OGNI IMPULSO DI CLOCK, VI È LO SCORRIMENTO DI UN BIT DA SINISTRA VERSO DESTRA

ANALISI E SINTESI DI UNA RETE SEQUENZIALE SINCRONA

- ESAMINARE PORTE E FLIP-FLOP ⇒ PARTI DALLO SCHEMA GATE-LEVEL: ANNOTA TUTTI I SEGNALE COMBINATORI: VANNO AGLI INGRESSI DEI FLIP-FLOP E QUINDI ESCONO DA ESSI
- RICAVARE LA TABELLA DELLE TRANSIZIONI ⇒ PER CIASCUNA CONFIGURAZIONE DI STATO CORRENTE S (CONTENUTO DEI FLIP-FLOP) E CIASCUN VALORE POSSIBILE DEGLI INGRESSI X, DETERMINA
 - LO STATO SUCCESSIVO S' (CHE VERRÀ CARICATO NEL FLIP-FLOP AL FRONTE DI CLOCK)
 - LE USCITE Y
- CONSTRUIRE IL DIAGRAMMA DEGLI STATI ⇒ OGNI NODO RAPPRESENTA UNO STATO S; OGNI ARCO, ETICHETATO CON X/Y, PORTA DA UNO STATO AL SUCCESSIVO.

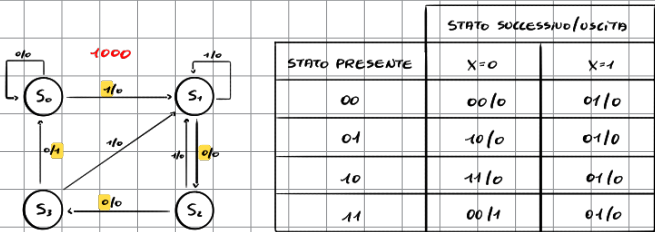
ESEMPIO DI SINTESI DI UNA RETE SEQUENZIALE SINCRONA



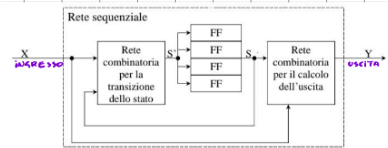
SI VOGLIA PROGETTARE UNA RETE LOGICA IN GRADO DI RICONOSCERE IN UNA SEQUENZA DI BIT IN INGRESSO LA STRINGA 1000 PONEENDO A 1 L'USCITA Z SOLO QUANDO SI ABBIA IL RICONOSCIMENTO DI TALE STRINGA

SI RICHIEDE:

- IL DIAGRAMMA DEGLI STATI;
- LA TABELLA DELLE TRANSIZIONI MEDIANTE L'USO DI FLIP-FLOP
- LA SINTESI DELLA RETE LOGICA PER LA TRANSIZIONE DI STATO E PER IL CALCOLO DELL'USCITA USANDO LE MAPPE DI KARNAUH



SCHEMA GENERALE RETE SEQUENZIALE



ESEMPIO PRATICO

NELLA PROGETTAZIONE DEL RICONOSCITORE DELLA STRINGA "1000" ABBIAMO 4 STATI. ⇒ SERVONO 2 FLIP-FLOP D (PERCHÉ 2=4)

IL BLOCCO COMBINATORIO DI TRANSIZIONE SARÀ UNA PICCOLA "ALU" LOGICA A PORTE CHE, DATI X E I DUE BIT DI STATO Q1, Q0, CALCOLA S1, S0

Ora dobbiamo progettare la «rete combinatoria per la transizione dello stato», la rete che «piloterà» le transizioni di stato dei flip-flop in accordo con il diagramma degli stati precedente

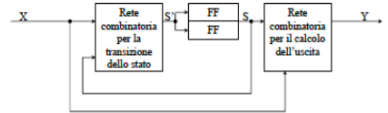
- INGRESSI X
- RETE COMBINATORIA PER LA TRANSIZIONE DI STATO (PRENDE IN INGRESSO IL VETTORE X E CALCOLA LO STATO SUCCESSIVO S')
- REGISTRO (BLOCCO RITARDA) È COMPOSTO DA TANTI FLIP-FLOP QUANTI SONO I BIT NECESSARI A CODIFICARE GLI STATI INTERNI. AL FRONTE DI CLOCK CARICA S', FORMANDO AL CICLO SUCCESSIVO LO STATO CORRENTE S
- RETE COMBINATORIA PER IL CALCOLO DELLE USCITE. USA X PER PRODURRE L'USCITA Y
- FEEDBACK. LE USCITE DEL REGISTRO S RITORNANO NELLA RETE DI TRANSIZIONE DI STATO, CHIUDENDO IL CICLO

FUNZIONE DI TRANSIZIONE DELLO STATO F

DATO L'INPUT E LO STATO S AD UN DATO Istante, CALCOLA LO STATO S' ALL'ISTANTE SUCCESSIVO S' = F(X, S)

- PARTIRE DALLA TABELLA DELLE TRANSIZIONI (STATO PRESENTE + INGRESSO ⇒ STATO SUCCESSIVO)
- PER OGNI BIT Si DELLO STATO SUCCESSIVO SCRIVERE L'EQUAZIONE BOOLEANA Si = fi(X, S)
- MINIMIZZARE fi (CON MAPPE DI KARNAUH O SOFTWARE) E REALIZZARLA CON PORTE LOGICHE

Nota: quando si usa un flip-flop D, occorre definire Di, S'i; Se si usasse un JK o un T, bisognerebbe invece tradurre S'i nella corrispondente combinazione di ingressi Ji, Ki, o Ti



RETE COMBINATORIA PER TRANSIZIONE STATO

STATO PRESENTE	STATO SUCCESSIVO/USCITA	
	X=0	X=1
00	00/0	01/0
01	10/0	01/0
10	11/0	01/0
11	00/1	01/0

LE DUE COLONNE "STATO SUCCESSIVO" SONO I BIT DEL PROSSIMO STATO A'B'

QUANDO HA "1" CON X=0 TORNIAMO A "00" CINETTIAMO Z=1

Q	Q(t)	Q(t+Z)
0	0	0
0	1	0
1	0	1
1	1	1

LA RETE COMBINATORIA PER LA TRANSIZIONE DELLO STATO DEVE FORNIRE AI 2 FLIP FLOP 6 GLI

INGRESSI OPPORTUNI CHE «PILOTANO» I FLIP FLOP A COMPIERE LE TRANSIZIONI DI STATO DEFINITE

DALLA TABELLA

DALLA TABELLA DI TRANSIZIONE ALLA "TABELLA DI PILOTAGGIO" DEI D-FF

- A E B RAPPRESENTANO LO STATO ATTUALE MEMORIZZATO IN CIASCUNO DEI 2 FLIP FLOP, A' E B' LO STATO SUCCESSIVO IN CUI VOGLIO ANDARE IN FUNZIONE DELL'INGRESSO X E DELLO STATO $A'=F_A(X,A)$ $B'=F_B(X,B)$

- LA TABELLA DI VERITA' DESCRIVE LE DUE FUNZIONI CHE VOGLIO REALIZZARE

- HA E HB SONO GLI INGRESSI CHE DEVO FORNIRE AI 2 FLIP FLOP PER

FARGLI FARE LE TRANSIZIONI DI STATO DESIDERANTE

A	B	X	A'	DA	B'	DB
0	0	0	0	0	0	0
0	0	1	0	0	1	1
0	1	0	1	1	0	0
0	1	1	0	0	1	1
1	0	0	1	1	1	1
1	0	1	0	0	1	1
1	1	0	0	0	0	0
1	1	1	0	0	1	1

ORA SI PUO' MINIMIZZARE CIASCUNA FUNZIONE HA(A,B,X) E HB(A,B,X) CON MAPPE DI KARNAUSH

HA -> SARA' "1" SOLO NELLE RIGHE IN CUI A'=1 OVVERO (A,B,X)=(0,1,0) OPPURE (1,0,0)

HB -> SARA' "1" QUANDO (A,B,X)=(0,0,1), (0,1,1), (1,0,1), (1,0,1), (1,1,1)

HA=A' HB=B'

CON D-FF -> SEMPRE DA=A' E DB=B'

CON JK-FF o T-FF -> CHEVI PARE UNA MAPPAUTURA JK o T -> STATO SUCCESSIVO

RIGA AB=00 -> X=0 STATO SUCCESSIVO=00 A'=0 B'=0
X=1 STATO SUCCESSIVO=01 A'=0 B'=1

RIGA AB=10 -> X=0 STATO SUCCESSIVO=11 A'=1 B'=1
X=1 STATO SUCCESSIVO=01 A'=0 B'=1

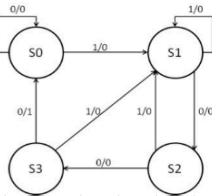
RIGA AB=01 -> X=0 STATO SUCCESSIVO=10 A'=1 B'=0
X=1 STATO SUCCESSIVO=01 A'=0 B'=1

RIGA AB=11 -> X=0 STATO SUCCESSIVO=00 A'=0 B'=0
X=1 STATO SUCCESSIVO=01 A'=0 B'=1

PER TROVARE LO STATO SUCCESSIVO A' E B' CHE DIPENDE DAI GRAFI

Da grafo stati a tabella pilotaggio flip-flop

- E' semplice trovare la corrispondenza fra nodi del grafo (diagramma degli stati) e le righe della tabella di pilotaggio dei flip-flop
- Nel caso del flip-flop D è anche molto semplice trovare quali sono gli ingressi DA e DB che devo fornire ai flip-flop per ottenere la transizione desiderata (il flip-flop D semplicemente «memorizza/ricopia» il valore dell'ingresso)



A	B	X	A'	DA	B'	DB
0	0	0	0	0	0	0
0	0	1	0	0	1	1
0	1	0	1	1	0	0
0	1	1	0	0	1	1
1	0	0	1	1	1	1
1	0	1	0	0	1	1
1	1	0	0	0	0	0
1	1	1	0	0	1	1

Sintesi rete transizione dello stato

Per sintetizzare la rete combinatoria «minima» per la transizione dello stato che «pilota» gli ingressi dei due flip-flop devo semplicemente sintetizzare due reti logiche combinatorie con le mappe di Karnaugh. Le reti che producono in uscita DA e DB dati in ingresso A, B ed X.

A	B	X	A'	DA	B'	DB
0	0	0	0	0	0	0
0	0	1	0	0	1	1
0	1	0	1	1	0	0
0	1	1	0	0	1	1
1	0	0	1	1	1	1
1	0	1	0	0	1	1
1	1	0	0	0	0	0
1	1	1	0	0	1	1

Sintesi rete per transizione dello stato

Sintesi delle due reti logiche combinatorie con le mappe di Karnaugh. Le reti che producono in uscita DA e DB dati in ingresso A, B ed X

AB	00	01	11	10
X=0		1		1
X=1				

$$D_A = \bar{A}B\bar{X} + A\bar{B}\bar{X}$$

AB	00	01	11	10
X=0				1
X=1	1	1	1	1

$$D_B = A\bar{B} + X$$

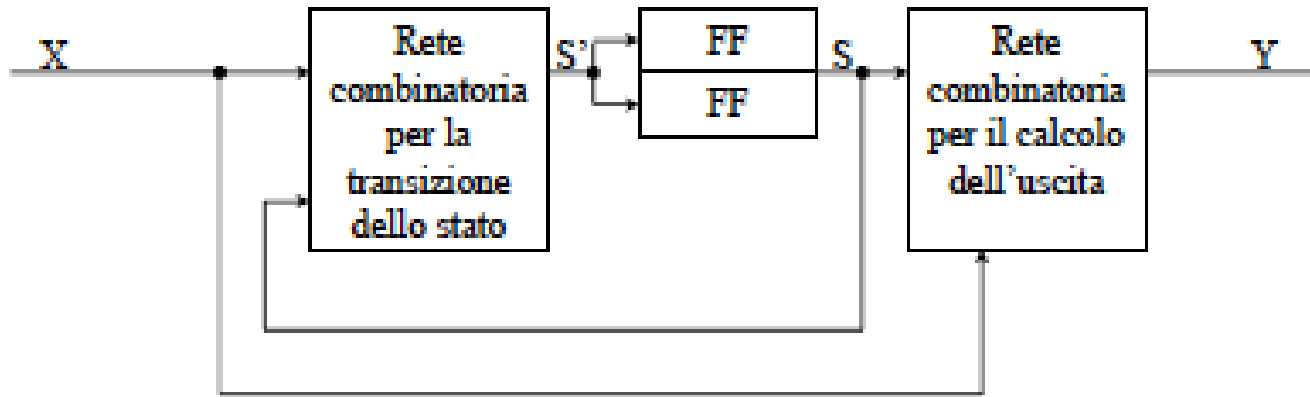
Sintesi rete combinatoria per uscita

A	B	X	Z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

In questo caso la sintesi della rete combinatoria è così semplice che non servono neppure le mappe di Karnaugh. Z vale 1 solo in una configurazione

$$Z = A\bar{B}\bar{X}$$

Schema logico della rete logica finale



- Per ottenere lo schema logico finale è sufficiente sostituire nello schema logico generale di cui sopra lo schema delle due reti che costituiscono la rete per la transizione dello stato:

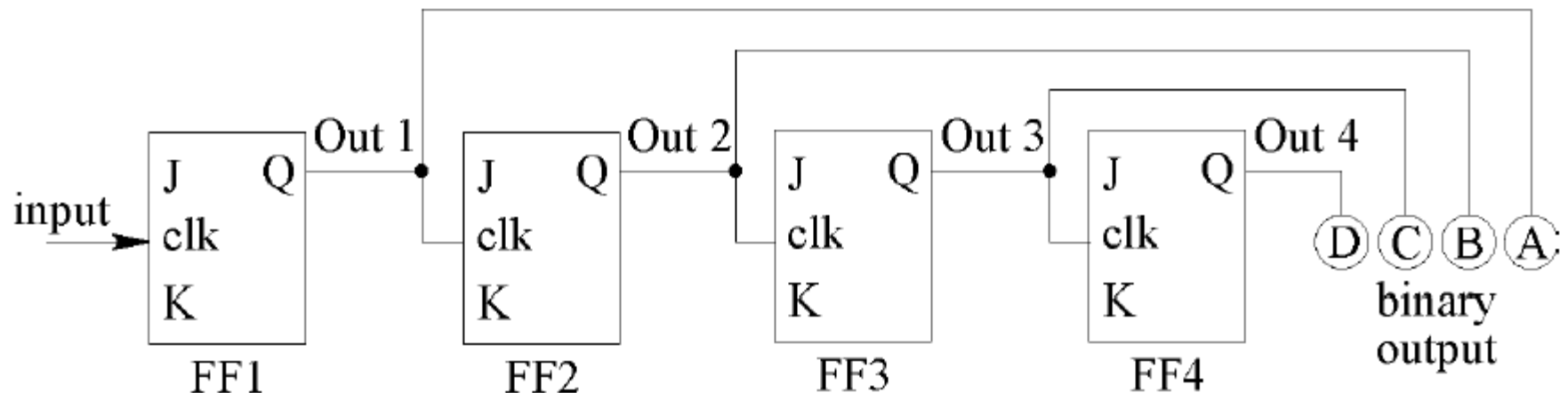
$$D_A = A' B X' + A B' X'$$

$$D_B = A B' + X$$

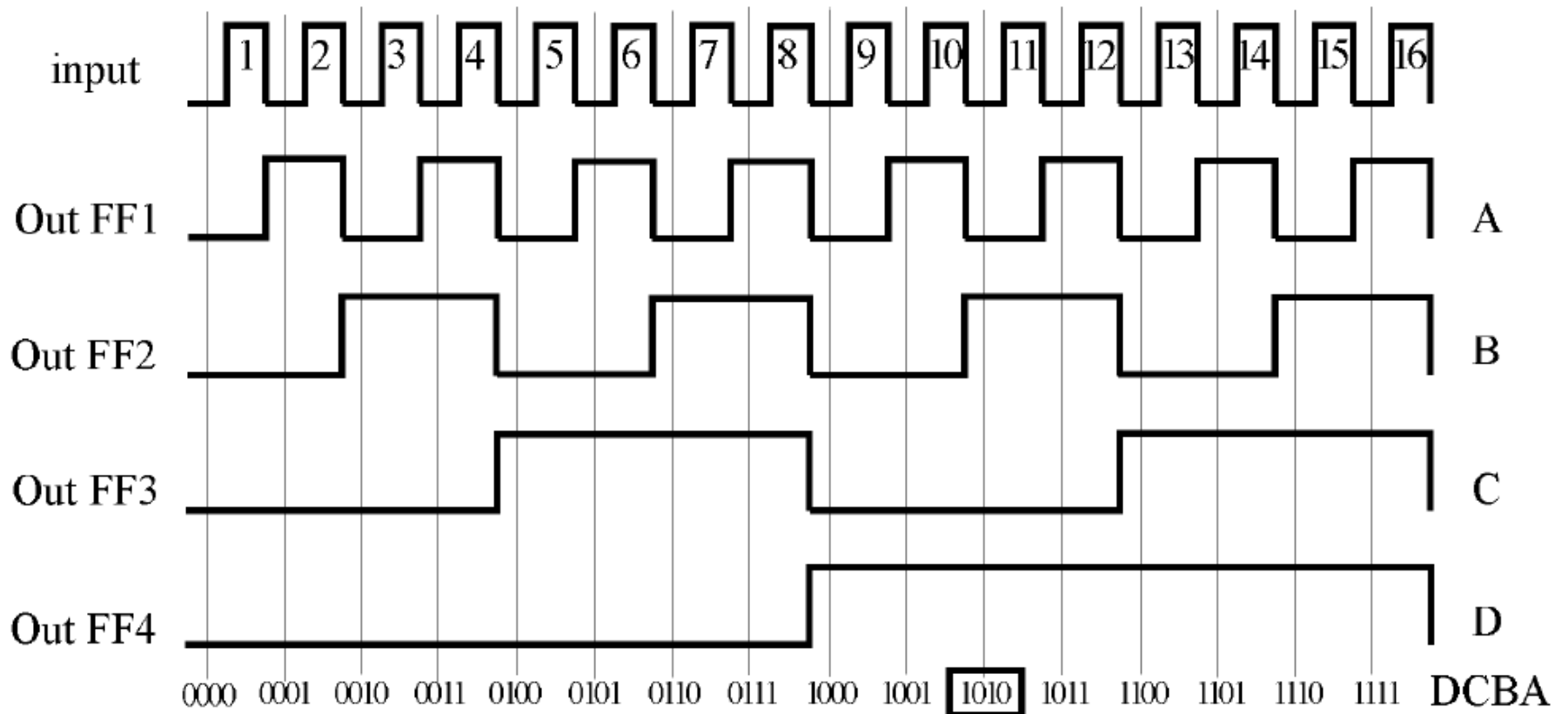
- E lo schema logico della rete per il calcolo dell'uscita Z

Reti sequenziali notevoli: i contatori

- Ad ogni segnale di clock, il segnale Q del flip flop i-esimo commuta con frequenza dimezzata rispetto a quella del flip flop (i-1)-esimo
- Le uscite dei flip flop (indicate con A, B, C, D) realizzano un contatore binario da 0 a 15



Funzionamento del contatore



Reti sequenziali notevoli: le memorie

